

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Deliverables and Grading Policy

Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	Java
GUI library (strongly encouraged)	Java swing
IDE (Integrated Development Environment)	Eclipse
xUnit framework (e.g., JUnit for Java)	JUnit
Programming style guide (must read it carefully)	Google Java Style Guide
Project hosting site	Github.com
Other decisions if applicable	

Sample programming style guides:

- Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

2. Unit testing (4 points) -> JUnit5

The screenshot shows the Eclipse IDE interface. The main editor displays the file `SolitaireSettingsTest.java` with the following code:

```
1 package solitary2;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4
5 import org.junit.jupiter.api.Test;
6
7 public class SolitaireSettingsTest {
8
9     @Test
10    public void testDrawThreeMode() {
11
12        SolitaireSettings settings = new SolitaireSettings();
13
14        assertEquals(
15            "Draw 3",
16            settings.getDrawMode(true)
17        );
18    }
19
20    @Test
21    public void testHintsEnabled() {
22
23        SolitaireSettings settings = new SolitaireSettings();
24
25        assertEquals(
26            "Hints On",
27            settings.getHintSetting(true)
28        );
29    }
30 }
```

The left sidebar shows the Package Explorer with the project structure and the JUnit 5 test results. The test results show two tests passed: `testDrawThreeMode()` (0.011 s) and `testHintsEnabled()` (0.001 s). The bottom status bar indicates "Runs: 2/2", "Errors: 0", and "Failures: 0".

```

package solitary2;
public class SolitaireSettings {
    public String getDrawMode(boolean drawThree) {
        if (drawThree) {
            return "Draw 3";
        } else {
            return "Draw 1";
        }
    }
    public String getHintSetting(boolean hintsEnabled) {
        if (hintsEnabled) {
            return "Hints On";
        } else {
            return "Hints Off";
        }
    }
}

```

Main.java:

```

package solitary2;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
public class SolitaireSettingsTest {
    @Test
    public void testDrawThreeMode() {
        SolitaireSettings settings = new SolitaireSettings();
        assertEquals(
            "Draw 3",
            settings.getDrawMode(true)
        );
    }
    @Test
    public void testHintsEnabled() {
        SolitaireSettings settings = new SolitaireSettings();
        assertEquals(
            "Hints On",
            settings.getHintSetting(true)
        );
    }
}

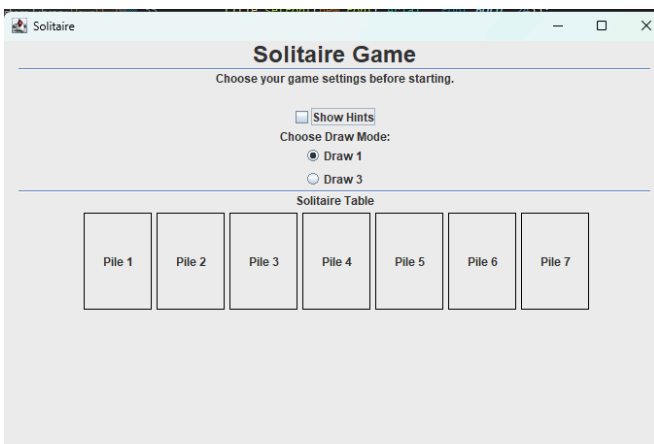
```

Test.java

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.



```
package solitaire2;
import java.awt.Component;
import java.awt.Dimension;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.ButtonGroup;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JSeparator;
import javax.swing.SwingConstants;
public class SolitaireGUI {
    public static void main(String[] args) {
        // Creates the main window
        JFrame frame = new JFrame("Solitaire");
        frame.setSize(700, 500);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Creates the main panel
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        // Creates the title text
        JLabel title = new JLabel("Solitaire Game");
```

```

title.setFont(new Font("Arial", Font.BOLD, 24));
title.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(title);
// Creates a line under the title
JSeparator line = new JSeparator();
line.setMaximumSize(new Dimension(650, 10));
panel.add(line);
// Creates instruction text
JLabel instructions =
    new JLabel("Choose your game settings before starting.");
instructions.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(instructions);
// Adds space
panel.add(Box.createRigidArea(new Dimension(0, 20)));
// Creates checkbox
JCheckBox hints = new JCheckBox("Show Hints");
hints.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(hints);
// Creates text for radio buttons
JLabel drawLabel = new JLabel("Choose Draw Mode:");
drawLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(drawLabel);
// Creates radio buttons
JRadioButton drawOne = new JRadioButton("Draw 1");
JRadioButton drawThree = new JRadioButton("Draw 3");
drawOne.setAlignmentX(Component.CENTER_ALIGNMENT);
drawThree.setAlignmentX(Component.CENTER_ALIGNMENT);
// Groups radio buttons
ButtonGroup drawGroup = new ButtonGroup();
drawGroup.add(drawOne);
drawGroup.add(drawThree);
// Default option
drawOne.setSelected(true);
panel.add(drawOne);
panel.add(drawThree);
// Creates another line
JSeparator secondLine = new JSeparator();
secondLine.setMaximumSize(new Dimension(650, 10));
panel.add(secondLine);
// Creates tableau label
JLabel tableauLabel = new JLabel("Solitaire Table");
tableauLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(tableauLabel);
// Creates panel for card piles
JPanel cardPanel = new JPanel();
// Creates seven simple card piles
for (int i = 1; i <= 7; i++) {
    JLabel pile = new JLabel(
        "Pile " + i,
        SwingConstants.CENTER
    );
    pile.setPreferredSize(new Dimension(70, 100));
    pile.setBorder(
        BorderFactory.createLineBorder(java.awt.Color.BLACK)
    );
    cardPanel.add(pile);
}
panel.add(cardPanel);
// Creates Start Game button
JButton startButton = new JButton("Start Game");
startButton.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(startButton);

```

```

// Creates status text
JLabel status =
    new JLabel("Choose settings and start the game.");
status.setAlignmentX(Component.CENTER_ALIGNMENT);
panel.add(status);
// Runs when Start Game is clicked
startButton.addActionListener(e -> {
    SolitaireSettings settings =
        new SolitaireSettings();
    String drawMode =
        settings.getDrawMode(
            drawThree.isSelected()
        );
    String hintSetting =
        settings.getHintSetting(
            hints.isSelected()
        );
    status.setText(
        "Game started: "
        + drawMode
        + " | "
        + hintSetting
    );
});
// Adds panel to window
frame.add(panel);
// Centers the window
frame.setLocationRelativeTo(null);
// Shows the window
frame.setVisible(true);
}
}

```